FLASK WEB DEVELOPMENT

Module 3

Forms and Request Handling

HTML forms • GET & POST • the request object • file uploads • validation

Input Forms & Validation

File Uploads & Responses

Prof. Kuntal Pramanik | BCA / MCA Dept, Parul University

MODULE ROADMAP

What We'll Cover

01

HTML Forms

Building forms with inputs, labels, and submit controls

02

HTTP Methods

How GET and POST carry data to the server

03

Request Object

Reading submitted data with Flask's request

04

Files & Validation

Handling uploads safely and validating input

Anatomy of an HTML Form

A form is the browser's way of collecting user input and packaging it for the server.

```
<form action="/register" method="POST">
  <label>Name</label>
  <input type="text" name="username">

  <label>Email</label>
  <input type="email" name="email">

  <label>Resume</label>
  <input type="file" name="resume">

  <button type="submit">Submit</button>
</form>
```

1. **action** — URL that receives the submitted data
2. **method** — HTTP verb used — GET or POST
3. **name** — Key used to read the value on the server
4. **enctype** — multipart/form-data required for file inputs

GET vs POST

The form's method attribute decides how data travels to the server.

GET

- Appends data to the URL as a query string
- Visible in the address bar & browser history
- Data length is limited
- Best for search, filters, navigation
- Safe to bookmark or refresh

POST

- Sends data in the request body
- Not visible in the URL
- No practical size limit
- Best for login, registration, uploads
- Not cached or bookmarked by default

Handling GET Requests

```
@app.route("/search")
def search():
    query = request.args.get("q", "")
    results = run_search(query)
    return render_template(
        "results.html",
        results=results
    )
```

example.com/search?q=flask

request.args

- Reads values from the query string (?key=value)
- request.args.get("q") avoids KeyError if missing
- Great for filters, search boxes, pagination
- Data is visible and shareable via the URL

Handling POST Requests

```
@app.route("/register",
           methods=["GET", "POST"])
def register():
    if request.method == "POST":
        name = request.form["username"]
        email = request.form["email"]
        save_user(name, email)
        return redirect("/success")
    return render_template("register.html")
```

request.form

- Reads values from the submitted request body
- Route must explicitly allow methods=["POST"]
- Always check request.method before branching
- Used for logins, registrations, and form submissions

Flask's request Object

Imported from flask, request carries everything the client sent — for the current request only.

`request.method`

The HTTP verb used — "GET" or "POST"

`request.form`

Submitted form fields (POST body)

`request.args`

Query-string parameters (?key=value)

`request.values`

Combined form + args, read-only

`request.files`

Uploaded files as FileStorage objects

`request.cookies`

Cookies sent by the browser

Reading Submitted Data

```
@app.route("/contact", methods=["POST"])
def contact():
    name = request.form.get("name")
    email = request.form.get("email")
    topic = request.args.get("ref", "site")

    if not name or not email:
        flash("All fields are required")
        return redirect("/contact")

    send_message(name, email, topic)
    return render_template("thanks.html")
```

- Use .get("key") over ["key"] to avoid crashing on missing fields
- Pass a default: .get("topic", "site")
- request.form is a dict-like ImmutableMultiDict
- Combine with request.method to branch GET vs POST logic

How File Uploads Work

File inputs need a different form encoding — and a different read on the server.

1

Form

`enctype="multipart/form-data"`

Required so the browser sends binary file data

2

Browser

`Uploads the file in the request body`

Alongside any other form fields

3

Flask

`request.files["field_name"]`

Returns a FileStorage object for the file

4

Server

`file.save(path)`

Writes it to the upload folder on disk

Saving Uploaded Files Safely

```
from werkzeug.utils import secure_filename

app.config["UPLOAD_FOLDER"] = "uploads/"
ALLOWED = {"png", "jpg", "jpeg", "pdf"}

@app.route("/upload", methods=["POST"])
def upload():
    file = request.files.get("resume")
    if file and allowed(file.filename):
        name = secure_filename(file.filename)
        path = os.path.join(
            app.config["UPLOAD_FOLDER"], name)
        file.save(path)
        return "Uploaded successfully"
    return "Invalid file", 400
```

`secure_filename()`

Strips path characters & unsafe names before saving

`allowed(filename)`

Checks the extension against a whitelist

`UPLOAD_FOLDER`

A dedicated, writable directory — never the app root

Input Validation

Never trust submitted data — validate on the server even if the browser already checked it.

Client-Side

- HTML attributes: required, type, pattern, maxlength
- Instant feedback, better UX
- Easy to bypass — not a security control

Server-Side

- Checked in the Flask route on every request
- Validates presence, type, length, format
- The only validation that can be trusted

Common checks: required fields · data type · length limits · email/pattern format · file type & size

Server-Side Validation Example

```
@app.route("/register", methods=["POST"])
def register():
    errors = []
    name = request.form.get("username", "").strip()
    email = request.form.get("email", "").strip()
    age = request.form.get("age", "")

    if not name or len(name) < 3:
        errors.append("Username must be at least 3 characters")
    if "@" not in email:
        errors.append("Enter a valid email address")
    if not age.isdigit() or not (0 < int(age) < 120):
        errors.append("Age must be a number between 1 and 119")

    if errors:
        return render_template("register.html", errors=errors)

    save_user(name, email, age)
    return redirect("/success")
```

LAB WORK

Hands-On Lab

Two connected labs — build the form, then handle what it sends.

Lab 1

Create Input Forms with Validation

Build an HTML form and validate its fields on the server before accepting the data.

Lab 2

Handle File Uploads & Form Responses

Extend the form to accept a file, save it safely, and return a proper response.

LAB 1

Create Input Forms with Validation

1. **Design the form** Create an HTML template with name, email, and age fields plus a submit button.
2. **Add the route** Create a Flask route that accepts GET (show form) and POST (process it).
3. **Read the data** Use request.form.get() to pull each field inside the POST branch.
4. **Validate server-side** Check required fields, length, and format; collect any errors found.
5. **Return feedback** Re-render the form with error messages, or redirect on success.

LAB 2

Handle File Uploads & Form Responses

1. **Update the form** Add enctype="multipart/form-data" and a file input for the upload.
2. **Configure Flask** Set UPLOAD_FOLDER and a whitelist of ALLOWED_EXTENSIONS.
3. **Read the file** Access it with request.files["field_name"] in the POST branch.
4. **Save it safely** Clean the filename with secure_filename() before calling file.save().
5. **Respond to the user** Show a success message or redirect to a confirmation page.

KEY TAKEAWAYS

Summary

- HTML forms collect user input and submit it via GET or POST
- GET sends data in the URL; POST sends it in the request body
- request.form, request.args, and request.files read submitted data
- Uploads need multipart/form-data and secure_filename() before saving
- Always validate on the server — client-side checks can be bypassed